

PROGRAM 1

AIM - Write a Program for finding roots of $f(x)=0$ by bisection method.

Software used - Online C++ compiler

Algorithm –

1. Define a function `func(x)` to represent the equation $(x^3 - x^2 + 2)$.
2. Define the `root\_through\_bisection` function to find the root in the interval $[x, y]$.
3. Check if the initial interval $[x, y]$ contains a root by ensuring $\text{func}(x) * \text{func}(y) < 0$.
4. Initialize `middle` as the midpoint of $[x, y]$.
5. Repeat the process while the difference between `y` and `x` is greater than or equal to the fined tolerance (`EPSILON`).
6. Calculate $\text{middle} = (x + y) / 2$.
7. If $\text{func}(\text{middle}) == 0$, return `middle` as the root.
8. If $\text{func}(\text{middle}) * \text{func}(x) < 0$, update $y = \text{middle}$, otherwise update $x = \text{middle}$.
9. Return `middle` as the approximate root.
10. In `main()`, call `root_through_bisection(-500, 100)`
11. print the result.

Source code –

```
#include <iostream>
using namespace std;
#define EPSILON 0.001
double func(double x)
{
    return x*x*x - x*x + 2;
}
double root_through_bisection(double x, double y)
{
    if (func(x) * func(y) >= 0)
    {
        cout << "The initial interval does not contain a root." << endl;
        return 0.0;
    }
```

```

double middle = x;
while (y - x >= EPSILON)
{
    middle = (x + y) / 2;
    if (func(middle) == 0.0)
    {
        return middle;
    }
    else if (func(middle) * func(x) < 0)
    {
        y = middle;
    }
    else
    {
        x = middle;
    }
}
return middle;
}

int main()
{
    cout << "root of the given equation is: " << root_through_bisection(-500, 100) << endl;
    return 0;
}

```

Output -

```
root of the given equation is: -1.0004
```

```
=== Code Execution Successful ===
```

PROGRAM 2

AIM – Write a Program for finding roots of $f(x)=0$ Newton Raphson method

Software used - Online C++ compiler

Algorithm –

1. Define the function: $\text{func}(x) = x^3 - x^2 + 2$
2. Define the derivative: $\text{derivate_func}(x) = 3x^2 - 2x$
3. Initialize guess: Set a to the initial guess value (e.g., -10)
4. Compute initial step size: $h = \text{func}(a) / \text{derivate_func}(a)$
5. Check convergence: While $|\text{abs}(h)| \geq \text{EPSILON}$:
 - Update guess: Set $a = a - h$
 - Compute new step size: $h = \text{func}(a) / \text{derivate_func}(a)$
6. Return root: Return a as the approximated root

Source code –

```
#include <iostream>

using namespace std;

#define EPSILON 0.001

double func(double x)
{
    return x * x * x - x * x + 2;
}

double derivate_func(double x)
{
    return 3 * x * x - 2 * x;
}

double NewtonRaphson(double a)
{
    double h = func(a) / derivate_func(a);
    while (abs(h) >= EPSILON)
    {
        h = func(a) / derivate_func(a);
        a = a - h;
    }
}
```

```
    }  
    return a;  
}  
int main()  
{  
    cout << "root of the given equation is: " << NewtonRaphson(-10) << endl;  
    return 0;  
}
```

Output -

```
root of the given equation is: -1  
  
=== Code Execution Successful ===
```

PROGRAM 3

AIM – Write a program Program for finding roots of $f(x)=0$ by secant method

Software used - Online C++ compiler

Algorithm -

1. Start.
2. Define the function $f(x) = x^3 + x^2 - 4$.
3. Input initial guesses x_1 and x_2 , tolerance, and maximum iterations.
4. Initialize `iterations` to 0.
5. Repeat until convergence or maximum iterations:
6. Calculate the next root approximation using the secant formula:
$$\text{rootApproximation} = x_2 - \frac{f(x_2) \cdot (x_2 - x_1)}{f(x_2) - f(x_1)}$$
7. Update x_1 to x_2 and x_2 to `rootApproximation`.
8. Increment `iterations`.
9. Check if maximum iterations are reached:
 - If yes, print "Maximum iterations reached" and exit loop.
10. Check if the absolute value of the function at `rootApproximation` is less than tolerance:
 - If yes, exit loop.
11. Return `rootApproximation` as the approximate root.
12. Print the approximate root.
13. End.

Source code –

```
#include <iostream>

#include <cmath>

using namespace std;

double Function(double x)
{
    return (x * x * x) + (x * x) - 4;
}

double secantMethod(double initialGuess1, double initialGuess2, double tolerance, int
maxIterations)
{
    double rootApproximation;

    int iterations = 0;

    do
    {
        rootApproximation = initialGuess2 - (Function(initialGuess2) * (initialGuess2 -
```

```

        initialGuess1)) / (Function(initialGuess2) - Function(initialGuess1));
    initialGuess1 = initialGuess2;
    initialGuess2 = rootApproximation;
    iterations++;
    if (iterations >= maxIterations)
    {
        cout << "Maximum iterations reached." << endl;
        break;
    }
} while (abs(Function(rootApproximation)) > tolerance);
return rootApproximation;
}

int main()
{
    double initialGuess1, initialGuess2, tolerance, root;
    int maxIterations;
    cout << "Enter initial guess 1: ";
    cin >> initialGuess1;
    cout << "Enter initial guess 2: ";
    cin >> initialGuess2;
    cout << "Enter tolerance (e.g., 1e-6): ";
    cin >> tolerance;
    cout << "Enter maximum iterations: ";
    cin >> maxIterations;

    root = secantMethod(initialGuess1, initialGuess2, tolerance, maxIterations);
    cout << "Approximate root: " << root << endl;
    return 0;
}

```

Output –

```
Enter initial guess 1: 1
Enter initial guess 2: 1.2
Enter tolerance (e.g., 1e-6): 0.0001
Enter maximum iterations: 8
Approximate root: 1.3146

=== Code Execution Successful ===
```

PROGRAM 4

AIM – Write a program To implement Lagrange's Interpolation formula

Software used - Online C++ compiler

Algorithm -

1. Start.
2. Initialize `ax[MAX + 1]`, `ay[MAX + 1]` arrays, and variables `nr`, `dr`, `x`, `y = 0`, `i`, `j`, and `n`.
3. Print "Lagrange Interpolation by Sidharth".
4. Prompt user to enter the number of data points `n`.
5. Read the number of data points `n`.
6. Print "Enter the set of values (x and y)".
7. For each index `i` from `0` to `n-1`, do:
 - Read `ax[i]` and `ay[i]`.
8. Print "Enter the value of x for which value of y is wanted".
9. Read the value `x`.
10. For each index `i` from `0` to `n-1`, do:
 - Initialize `nr = 1`.
 - Initialize `dr = 1`.
 - For each index `j` from `0` to `n-1`, do:
 - If `j != i`, do:
 - Multiply `nr` by `(x - ax[j])`.
 - Multiply `dr` by `(ax[i] - ax[j])`.
11. Add `(nr / dr) \* ay[i]` to `y`.
12. Print "When x = ", followed by the value of `x` formatted to 3 decimal places.
13. Print ", y = ", followed by the value of `y` formatted to 3 decimal places.
14. End.

Source code –

```
#include <iostream>

#include <iomanip>

using namespace std;

#define MAX 100

int main()
{
    float ax[MAX + 1], ay[MAX + 1], nr, dr, x, y = 0;
    int i, j, n;

    cout << "Lagrange Interpolation by Sidharth" << endl;

    cout << "Enter the number of data points (n): ";

    cin >> n;
```



```

cout << "Enter the set of values (x and y): " << endl;
for (i = 0; i < n; i++) {
    cin >> ax[i] >> ay[i];
}

cout << "Enter the value of x for which value of y is wanted: ";
cin >> x;
for (i = 0; i < n; i++) {
    nr = 1;
    dr = 1;
    for (j = 0; j < n; j++) {
        if (j != i) {
            nr *= (x - ax[j]);
            dr *= (ax[i] - ax[j]);
        }
    }
    y += (nr / dr) * ay[i];
}

cout << "When x = " << fixed << setprecision(3) << x;
cout << ", y = " << fixed << setprecision(3) << y << endl;
return 0;
}

```

Output –

```

Lagrange Interpolation by Sidharth
Enter the number of data points (n): 3
Enter the set of values (x and y):
1 1
2 4
3 9
Enter the value of x for which value of y is wanted: 2.5
When x = 2.500, y = 6.250

=== Code Execution Successful ===

```

PROGRAM 5

AIM – Write a program To implement Newton's Divided Difference formula

Software used - Online C++ compiler

Algorithm –

1. Input Observation Count: Read the number of observations (n).
2. Input X Values: Read n values of x .
3. Input Y Values: Read n values of y .
4. Input Desired X Value: Read the value of k for which the function $f(k)$ is to be evaluated.
5. Initialize the Result: Set the initial value of f to $y[0]$ (first value of y).
6. Start Forward Difference Calculation:
7. Repeat until only one value remains:
 - Compute Differences: Calculate the forward differences for y using the formula $p[i] = (y[i+1] - y[i]) / (x[i+1] - x[i])$.
 - Update Y Array: Update $y[i]$ with the calculated differences $p[i]$.
 - Calculate Product for Polynomial: Compute $f1$ as the product of $(k - x[i])$ for each i from 0 to j .
 - Update Result: Add the term $y[0] * f1$ to the accumulated result $f2$.
 - Reduce the Number of Points: Decrease n and increment j for the next order of differences.
8. Compute Final Result: Add $f2$ to f to get the final interpolated value.
9. Output Result: Print $f(k)$ as the interpolated result.

Source code –

```
#include <iostream>

using namespace std;

int main()
{
    int x[10], y[10], p[10];
    int k, f, n, i, j = 1;
    float f1 = 1, f2 = 0;

    cout << "Enter the number of observations: ";
    cin >> n;

    cout << "Enter the different values of x: ";
    for (i = 0; i < n; i++) {
        cin >> x[i];
```

```

    }

    cout << "Enter the corresponding values of y: ";
    for (i = 0; i < n; i++) {
        cin >> y[i];
    }

    cout << "Enter the value of 'k' in f(k) you want to evaluate: ";
    cin >> k;

    f = y[0];
    do {
        for (i = 0; i < n - 1; i++) {
            p[i] = (y[i + 1] - y[i]) / (x[i + 1] - x[i]);
            y[i] = p[i];
        }
        f1 = 1;
        for (i = 0; i < j; i++) {
            f1 *= (k - x[i]);
        }
        f2 += (y[0] * f1);
        n--;
        j++;
    }
    while (n > 1);
    f += f2;

    cout << "f(" << k << ") = " << f << endl;
    return 0;
}

```

Output –

```

Enter the number of observations: 3
Enter the different values of x: 1 2 3
Enter the corresponding values of y: 1 8 27
Enter the value of 'k' in f(k) you want to evaluate: 2.5
f(2) = 8

```

PROGRAM 6

AIM – Write a program for solving numerical integration by Trapezoidal rule

Software used - Online C++ compiler

Algorithm –

1. Start
2. Input: Read values x_0 , x_n , and n (number of subintervals).
3. Compute: Calculate step size h using the formula $(x_n - x_0) / n$.
4. Initialize: Set s to $y(x_0) + y(x_n)$ where $y(x)$ is the function $1 / (1 + x * x)$.
5. Loop: For each integer i from 1 to $n - 1$:
6. Update: Add $2 * y(x_0 + i * h)$ to s .
7. Calculate: Compute the integral value as $(h / 2) * s$.
8. Output: Print the integral value with 4 decimal places.
9. End

Flowchart –

Source code –

```
#include <iostream>

#include <iomanip>

#include <cmath>

float y(float x) {
    return 1 / (1 + x * x);
}

int main() {
    float x0, xn, h, s;

    int n;

    std::cout << "Enter x0, xn, and number of subintervals: ";
    std::cin >> x0 >> xn >> n;

    h = (xn - x0) / n;
    s = y(x0) + y(xn);

    for (int i = 1; i <= n - 1; i++) {
        s += 2 * y(x0 + i * h);
    }

    std::cout << "Value of integral is " << std::setw(10)
        << std::setprecision(4)
        << (h / 2) * s << std::endl;

    return 0;
}
```

Output –

```
Enter x0, xn, and number of subintervals: 0 1 4
Value of integral is      0.7828

=== Code Execution Successful ===
```

PROGRAM 7

AIM – Write a program for solving numerical integration by Simpson 1/3 rule

Software used - Online C++ compiler

Algorithm –

1. Start
2. Read x_0 , x_n , n .
3. Check if n is even. If not, (exit) with an error message.
4. Compute h as $(x_n - x_0) / n$.
5. Initialize s to $y(x_0) + y(x_n)$.
6. Sum terms $4 * y(x_0 + i * h)$ for odd i from 1 to $n-1$.
7. Sum terms $2 * y(x_0 + i * h)$ for even i from 2 to $n-2$.
8. Calculate the integral as $(h / 3) * s$.
9. Print the integral value formatted to 4 decimal places.
10. End

Flowchart –

Source code –

```
#include <iostream>

#include <iomanip>

#include <cmath>

float y(float x) {
    return 1 / (1 + x * x);
}

int main() {
    float x0, xn, h, s;

    int n;

    std::cout << "Enter x0, xn, and number of subintervals: ";

    std::cin >> x0 >> xn >> n;

    if (n % 2 == 0) {
        std::cerr << "Number of subintervals must be odd for Simpson's rule." << std::endl;
        return 1;
    }

    h = (xn - x0) / n;

    s = y(x0) + y(xn) + 4 * y(x0 + h);

    for (int i = 3; i <= n - 1; i += 2) {
        s += 4 * y(x0 + i * h) + 2 * y(x0 + (i - 1) * h);
    }

    std::cout << "Value of integral is " << std::setw(10)
        << std::setprecision(4)
        << (h / 3) * s << std::endl;

    return 0;
}
```

Output -

```
Enter x0, xn, and number of subintervals: 0 1 5
Value of integral is      0.6674

=== Code Execution Successful ===
```

PROGRAM 8

AIM – Write a program to implement numerical integration by Simpson 3/8 rule

Software used - Online C++ compiler

Algorithm –

1. Input the lower limit (x_0), upper limit (x_n), and the number of subintervals (n).
2. Calculate the step size (h) based on the range and subintervals.
3. Initialize the sum (s) by evaluating the function at the lower and upper limits.
4. Loop through each interval (i) from 1 to ($n-1$):
 - i. If (i) is divisible by 3, add (2 times) the function value at that point to the sum.
 - ii. Otherwise, add (3 times) the function value at that point to the sum.
5. Multiply the sum by the step size and constants of Simpson's 3/8 rule.
6. Output the final result as the approximate integral.

Flowchart –

Source code –

```
#include <iostream>

#include <cmath>

using namespace std;

float y(float x) {
    return 1 / (1 + x * x);
}

int main() {
    float result = 1;

    float x0, xn, h, s;

    int i, n;

    cout << "Enter the range - ";

    cout << "\nLower Limit x0 - ";

    cin >> x0;

    cout << "Upper Limit xn - ";

    cin >> xn;

    cout << "\nEnter number of subintervals - ";

    cin >> n;

    h = (xn - x0) / n;

    s = y(x0) + y(xn);

    for (i = 1; i < n; i++) {
        if (i % 3 == 0) {
            s += 2 * y(x0 + i * h);
        } else {
            s += 3 * y(x0 + i * h);
        }
    }

    result = s * 3 * h / 8;

    cout << "Value of the integral is \t" << result << endl;

    return 0;
}
```

Output -

```
Enter the range -  
Lower Limit x0 - 0  
Upper Limit xn - 1  
  
Enter number of subintervals - 6  
Value of the integral is    0.785396  
  
=== Code Execution Successful ===|
```

PROGRAM 9

AIM – Write a program to implement Unconstrained, Unimodal function minimisation using Fibonacci search method

Software used - Online C++ compiler

Algorithm –

1. Start
2. Define the function $f(x)$ to minimize, in this case $f(x)=x^2-6x+2$
3. Initialize the interval $[a, b]$, set the total number of Fibonacci iterations n , and calculate the initial interval length $L = b - a$.
4. For each iteration (from n down to 2):
5. Print the current values of a , b , and L .
6. Compute Fibonacci ratio using Fibonacci numbers $F(k-2)$ and $F(k)$.
7. Determine points x_1 and x_2 using the ratio and update the interval.
8. Evaluate function values at x_1 and x_2 , i.e., $f(x_1)$ and $f(x_2)$.
9. Update the interval: if $f(x_1) > f(x_2)$, update $a = x_1$, else update $b = x_2$.
10. Recompute the interval length and print the updated values.
11. Terminate after n iterations and print the estimated minimum point as the midpoint of the final interval $a+b \times \frac{1}{2}$
12. End

Flowchart –

Source code –

```
#include <iostream>

#include <cmath>

using namespace std;

double f(double x)
{
    return (x * x) - (6*x) + 2;
}

double Fibonacci(int n)
{
    if (n <= 1)
        return 1;

    int a = 1, b = 1;
    for (int i = 2; i <= n; i++)
    {
        int temp = a + b;
        a = b;
        b = temp;
    }

    return b;
}

int main()
{
    double a = 0.0;
    double b = 10.0;
    int n = 7;
    double L = b - a;
    int k = n;
    for (int k = n; k >= 2; --k)
    {
        cout << "for k = " << k << endl;
```

```
cout << "value of a = " << a << endl;
```

```
cout << "value of b = " << b << endl;
```

```
double z1 = Fibonacci(k - 2);
```

```
double z2 = Fibonacci(k);
```

```
double ratio = z1 / z2;
```

```
double delta = ratio * L;
```

```
cout << "delta =" << delta << endl;
```

```
double x1 = a + delta;
```

```
cout << "x1 = " << x1 << endl;
```

```
double x2 = b - delta;
```

```
cout << "x2 = " << x2 << endl;
```

```
double fx1 = f(x1);
```

```
double fx2 = f(x2);
```

```
cout << "f(x1) = " << fx1 << endl;
```

```
cout << "f(x2) = " << fx2 << endl;
```

```
if (fx1 > fx2)
```

```
{
```

```
    a = x1;
```

```
    cout << "fx1 > fx2.....hence new 'a' = " << a << endl;
```

```
}
```

```
else if (fx2 > fx1)
```

```
{
```

```
    b = x2;
```

```
    cout << "fx1 < fx2.....hence new 'b' = " << b << endl;
```

```
}
```

```
L = b - a;
```

```
cout << "updated value of L is : " << L << endl;
```

```
if (k == 2)
```

```

{
    cout << "minimum value of f(x) is at x = " << (a + b) / 2 << endl;
}

cout << "\n";

}

return 0;

}

```

Output -

```

for k = 7
value of a = 0
value of b = 10
delta =3.80952
x1 = 3.80952
x2 = 6.19048
f(x1) = -6.34467
f(x2) = 3.17914
fx1 < fx2....hence new 'b' = 6.19048
updated value of L is : 6.19048

```

```

for k = 6
value of a = 0
value of b = 6.19048
delta =2.38095
x1 = 2.38095
x2 = 3.80952
f(x1) = -6.61678
f(x2) = -6.34467
fx1 < fx2....hence new 'b' = 3.80952
updated value of L is : 3.80952

```

```

for k = 5
value of a = 0
value of b = 3.80952
delta =1.42857
x1 = 1.42857
x2 = 2.38095
f(x1) = -4.53061
f(x2) = -6.61678
fx1 > fx2....hence new 'a' = 1.42857
updated value of L is : 2.38095

```

```

for k = 4
value of a = 1.42857
value of b = 3.80952
delta =0.952381
x1 = 2.38095
x2 = 2.85714
f(x1) = -6.61678
f(x2) = -6.97959
fx1 > fx2....hence new 'a' = 2.38095
updated value of L is : 1.42857

```

```

for k = 3
value of a = 2.38095
value of b = 3.80952
delta =0.47619
x1 = 2.85714
x2 = 3.33333
f(x1) = -6.97959
f(x2) = -6.88889
fx1 < fx2....hence new 'b' = 3.33333
updated value of L is : 0.952381

```

```

for k = 2
value of a = 2.38095
value of b = 3.33333
delta =0.47619
x1 = 2.85714
x2 = 2.85714
f(x1) = -6.97959
f(x2) = -6.97959
updated value of L is : 0.952381
minimum value of f(x) is at x = 2.85714

```

PROGRAM 10

AIM – Write a program to implement Unconstrained, Unimodal function minimisation using Golden section search method

Software used - Online C++ complier

Algorithm –

1. Start
2. Initialize interval $[a, b]$, tolerance epsilon, and calculate the golden ratio ϕ .
3. Calculate delta for the current interval as $b - a\phi$ and $a\phi$.
4. Set points $x_1 = a + \delta$ and $x_2 = b - \delta$.
5. Evaluate function at x_1 and x_2 to get $f(x_1)$ and $f(x_2)$.
6. Update interval: if $f(x_1) > f(x_2)$, set $b = x_1$, else set $a = x_2$.
7. Repeat steps 2-5 until the interval length $L = b - a$ is less than epsilon.
8. Return minimum as the midpoint of the final interval $\frac{a + b}{2}$.
9. End

Flowchart –

Source code –

```
#include <iostream>

#include <cmath>

using namespace std;

// Function to minimize
double f(double x) {
    // Replace with your function here
    return x * x - 6 * x + 15;
}

// Golden Ratio value
const double phi = (1.0 + sqrt(5.0)) / 2.0;

int main() {
    cout << "phi = " << phi << endl;

    double a = 0.0; // Define your interval [a, b]
    double b = 4.0;

    double epsilon = 0.25; // Tolerance for convergence

    double x1, x2;

    double L = b - a;

    while (L > epsilon) {
        double delta = (b - a) / phi;

        cout << "delta = " << delta << endl;

        x2 = b - delta;
        x1 = a + delta;

        cout << "x1 = " << x1 << endl;
        cout << "x2 = " << x2 << endl;

        double f1 = f(x1);
        double f2 = f(x2);
```



```

cout << "f(x1) = " << f1 << endl;

cout << "f(x2) = " << f2 << endl;


if (f1 > f2) {
    b = x2;

    cout << "f(x1) > f(x2)....new b = " << b << endl;
} else {
    a = x1;

    cout << "f(x1) < f(x2)....new a = " << a << endl;
}

L = b - a;
}

// The minimum should be close to (a + b) / 2
double min_x = (a + b) / 2;
double min_value = f(min_x);


cout << "Minimum value: " << min_value << " at x = " << min_x << endl;

return 0;
}

```

Output -

```

phi = 1.61803
delta = 2.47214
x1 = 2.47214
x2 = 1.52786
f(x1) = 6.27864
f(x2) = 8.16718
f(x1) < f(x2)....new a = 2.47214
delta = 0.944272
x1 = 3.41641
x2 = 3.05573
f(x1) = 6.1734
f(x2) = 6.00311
f(x1) > f(x2)....new b = 3.05573
delta = 0.36068
x1 = 2.83282
x2 = 2.69505
f(x1) = 6.02795
f(x2) = 6.093
f(x1) < f(x2)....new a = 2.83282
Minimum value: 6.00311 at x = 2.94427

```